User Modeling and User-Adapted Interaction manuscript No.
(will be inserted by the editor)

A Model for Profile Management Through Inference in Trails

André Wagner · Jorge Luis Victória Barbosa · Débora Nice Ferrari Barbosa

Received: date / Accepted: date

Abstract Context-aware and ubiquitous systems have the challenge of implicitly collect relevant information about the users, and use this information to understand and predict their behaviour. This allows the applications to adapt themselves to the user, thus avoiding to overflow the user with inquires and information. This article presents a model that allows applications to register users' actions and infer profile information from these actions, using semantic interoperability and thus allowing different applications of different domains to be able to share information and infer a unified profile.

Keywords Profile Management · Ubiquitous Computing · Trails · Profile Inference

1 Introduction

Nowadays, most users interact with a variety of computing systems. The current tendency is the increased interconnectedness between these systems, since the internet is becoming more present and available anywhere. Recent research predict the full integration between these systems in a federation of computing systems. This federation can be described as Smart Environments (Heckmann, 2005).

André Wagner · Jorge Luis Victória Barbosa
Universidade do Vale do Rio dos Sinos (UNISINOS)
Unidade Acadêmica de Pesquisa e Pós-Graduação
Av. Unisinos, 950, São Leopoldo, Brazil
E-mail: andre.nho@gmail.com, jbarbosa@unisinos.br

Débora Nice Ferrari Barbosa
Universidade Feevale
Gr. Pesquisa em Informática na Educação
RS-239, 2755, Novo Hamburgo, Brazil
E-mail: deboranice@feevale.br

Weiser (1991) discussed a vision for the concept of smart envirnoments, where he presented a scenario in which the computer would become invisible and indistinguishable from its environment. In this scenario, interaction between humans and computers would not occur as usual – where the human devotes his full atention to the computer –, but would consist of small and quick interactions, usually in different devices and locations, and in many times using the "periphery" of the user's attention (Weiser & Brown, 1996), or without the user even realize he is interacting with a computer system. According to the author, "the most profound technologies are those that disappear. They weave themselves into the fabric of everyday life until they are indistinguishable from it." (Weiser, 1991). He asserts that the future of computing lies in integrating the computer to the real world, not the other way around, and he calls this new paradigm Ubiquitous Computing.

However, the increase of the computing resources available everytime and everywhere poses the risk of information overload on the user. To solve this problem, the concept of "calm technology" was created (Weiser & Brown, 1996). This means the transfer of information between the computer and the user must happen in the "perifery" of the user's attention, moving to the "center" of his attention in moments of urgency. Taking a car's engine as an example, when a person drives a car, his attention is focused on the road, and he barely notices the engine sound. However, if the engine emits a strange SOUND (That might indicate some kind of problem), the driver's attention turns to the engine.

If a system is to be minimally intrusive, it must undestand the situation of the user and his surroundings, and adapt its own behaviour according to this information. This information is called context, and systems capable of this kind of adaptation are called context-sensitive (Satyanarayanan, 2001; Dey, 2001).

Research in this area shows that is not only useful to know the context of the user when a service is being offered, but that the history of visited contexts can be a valuable information to help the application to adapt itself to the user. This history of visited contexts is called Trail (Silva et al., 2010). Altough the trail is usually tied to the location of a user, it has a broader concept, which includes activities performed, applications used, content accessed – among others – inside a specific context and timeframe (Silva et al., 2009).

For an application to adapt itself to a user effectively, it is not only important to know the history of contexts visited by a user, but to understand who the user is – that is, what are his characteristics. This kind of application organize their knowledge about a user in User Profiles. This information is stored in a format called User Model (Viviani et al., 2010).

Personalization means the individual adaptation of products, services and information. Personalization technology usually involve programs that learn patterns, habits and preferences of a user. The main goal of modern personalization systems is to offer the users what they want, without demanding them to inform it explicitly (Mulvenna et al., 2000). It is clear that this pur-

pose meets the ubiquitous computing objectivem that is, the development of a technology that "disappears".

For a system to be succeessful in obtaining the user's intent, it must know the user, the context where the user is inserted and the actions he took when he was previously in the same context or in similar contexts. Although there are some studies that seek to infer knowledge about a user from their trail, previous works operate within a specific context (such as Ubiquitous Education (Barbosa et al., 2011) or Ubiquitous Commerce (Franco et al., 2011)) or use a static user model (Levis et al., 2008).

A relevant aspect for environments with multiple independent systems is interoperability. Interoperability is the ability of two or more systems or components to exchange information and use the information that has been exchanged. (IEEE, 1991). In the context of customization, interoperability refers to the ability to access and interpret information derived from multiple heterogeneous sources and integrate this information into a common user model. Interoperability can be (i) syntactic, where the differences between the models are resolved user-level application, or (ii) semantic, where the differences between the models are resolved at the level of knowledge (Carmagnola & Cena, 2009).

Research indicates that it is desirable that future systems become increasingly invisible to their users (Weiser & Brown, 1996), and that a system can only become invisible if it is proactive, and can only be proactive by knowing the user (Satyanarayanan, 2001). For a system to know a user, he must know the user history, understand the context in which he is inserted, as well as data collected concerning him in the same context (or in a similar context), and be able to process this set of informations in a user profile.

In this context we are proposing the eProfile, a model for the automatic creation of profiles by inference of entities trails. This model allows the use of a single profile by several independent systems, offering the possibility of semantic interoperability between them.

In this model, the concept of entity is used instead of the classic concept of User. While usually the "user" means the person who is using the application, an entity refers to any object (physical or not) able to control an application. Thus, an entity can be a person, a moving object (like a car) or property (like a room), or even a system.

This article is organized as follows. Section 2 proposes the eProfile model. Section 3 discusses implementação aspects. Section 4 presents and experiment validating the model. Section 5 describes related work, showing the contributions of this model. Finally, a Section 6 presents the concluding remarks.

2 eProfile Model

Figure 1 shows the overview of eProfile, where each circle represents a component, and the arrows indicate the flow of data.

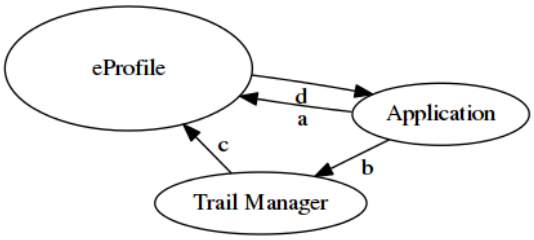

Fig. 1 Modelo básico do eProfile

The first component depicted is the **application**. The application, in this model, is any application that is being used directly by an entity. For an application to be considered suitable to use this model and able to use the functionality offered by eProfile, it needs to record their information on a trail manager supported by eProfile. The application must also implement a communication protocol to exchange messages with eProfile. If it follows these prerequisites, the application can be of any type and operate in any type of device that has communication with the Internet.

The second component depicted is a **trail manager**. A trail manager is capable of storing events held by an entity, putting them in chronological order and registering information relevant to the event (eg, the context in which the event was held). The eProfile manager supports any trail manager, since the trail manager communication protocol is implemented in eProfile.

The third component of Figure 1 represents the **eProfile**. The arrows indicate the flow of data. The communication starts when the application registers itself on eProfile, sending (i) the entity model used by the application and (ii) the rules for construction of this model (arrow "a"). From this moment on, the application starts recording the events on the trail manager (arrow "b"). The eProfile periodically retrieves this information from the trail manager and builds or reconfigures the profile entity (arrow "c"). When the application needs an updated profile, it retrieves this profile from eProfile (arrow "d").

Since the eProfile supports multiple applications, it is possible for different applications to use different trail managers. Therefore, the eProfile also supports communication with a number of trail managers, as is shown in Figure 2.

2.1 Architecture

Figure **??** shows the architecture of eProfile. The components "Application" and "Trail Manager" are the same as shown in Figure 1, while the "eProfile" service was expanded to show its inner workings.

Since the eProfile is a model that infers autonomously information about data stored in an ontology, and is organized into independent parts, it uses the technology of autonomous agents.

Each application builds its own entity model. The eProfile needs three pieces of information to be able to process the profiles: (i) the entity trail, (ii)

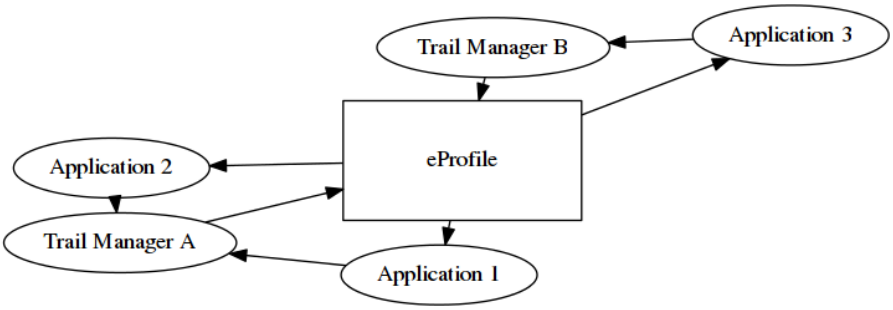

Fig. 2 eProfile connected to multiple trail managers

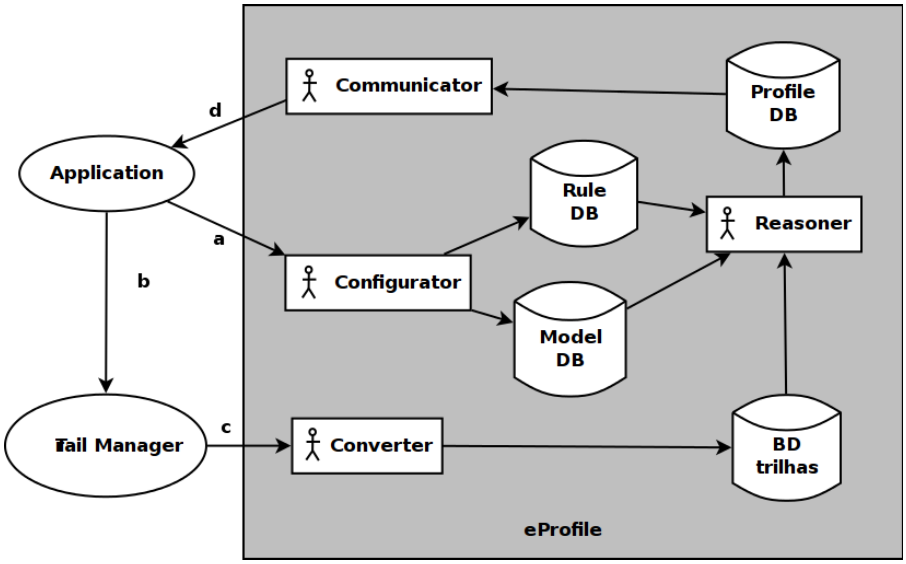

Fig. 3 eProfile Architecture

the entity model used by the application and (iii) the inference rules for the profile. This can be seen in Figure 3, where the agent "Reasoner" receives data from these three databases. The trail of the entity is loaded by the eProfile from the trail manager , while the other two informations are sent directly from the application to the eProfile. Thus, the inference rules defined by the application are used to infer the profile of the entity.

The arrows in Figure 3 indicate the direction of the data flow, where it can be noted two entry points and one exit point.

The first entry point (arrow "a") are the informations sent from the application to eProfile or, more specifically, to the **Configurator** agent. The following informations must be sent by the application:

- the entity model used by the application, which is the entity profile format. Once the entity trail is processed, the inferred data is stored and made available for application using this format;
- rules to infer the profile. These rules will be executed at regular intervals in order to update the entity profile. The Configurator receives the rules, validate them and insert them in **rule database**. Along with the rules, is specified the criticality of information. The system uses this information to determine how often the information will be inferred.

The second entry point (arrow "c") allows receiving the entities trails, that are fetched by the eProfile from the trail manager. Since eProfile can connect to different trail managers, (Figure 2), where each trail has one specific format, a conversion between the original format and the format used by eProfile is needed for the trails to be stored in a standardized manner. This work is performed by an agent named **Converter**, which is able to perform the conversion to the eProfile internal trail format.

The entity trail is then stored in an trail ontology, called **eOntoTrail**. This ontology has the basic classes supporting the construction of a trail, and they can be extended according to the specification of the trail manager which is being used.

Once in possession of the tracks, the model and rules, the **Reasoner** agent performs inference of the trail, generating a entity profile, which consists of an extension of a basic profile ontology named **eOntoProfile**. This ontology, just like the trail ontology, contains basic classes from which applications can specialize, using them to store their own information. The application can then connect to the eProfile and get its entity profile, whose data are sent by the **Communicator** agent (arrow "d").

2.2 Ontologies

As mentioned in Section 2.1, eProfile uses two ontologies: a trail ontology and a profile ontology. These are both basic ontologies, extended by subclasses according to the trail format sent by the trail manager (for the trail ontology) or the needs of the application (for the profile ontology).

The trail ontology (eOntoTrail) data, shown in Figure X, is fed by different supported trail manager, where the Converter agent is responsible for receiving data in the trail manager format and converting them to the eOntoTrail. Thus, the reasoner has a single ontology on which to reason.

Context-aware applications are interested in knowing **who**, **where**, **when** and **what** (that is, what the user is doing), so that they can discover why the situation is occurring (Abowd et al., 1999). Thus, a trail is represented by a set of events characterized by the following main classes: (i) *Entity*, the identity of the entity that is performing the action, which can optionally be linked to a profile, (ii) *Location*, the place where the action is taking place, (iii) *Time*, the date and time that the event occurred, (iv) *Participants*, the entities linked to this action, and (v) *Activity*, the action that the entity is performing (or

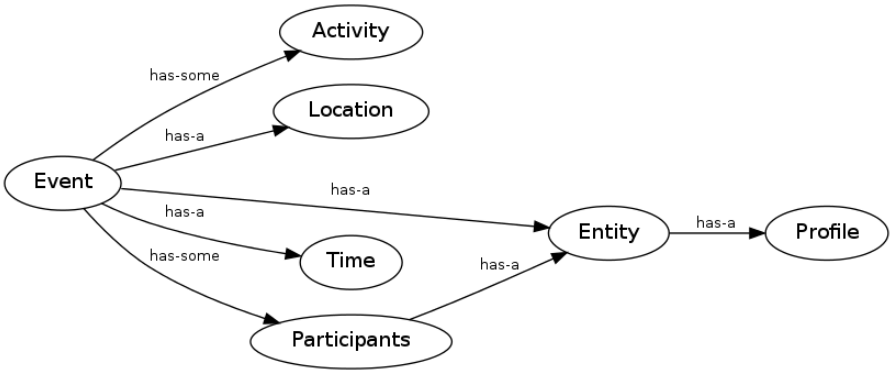

Fig. 4 eProfile Trail Ontology

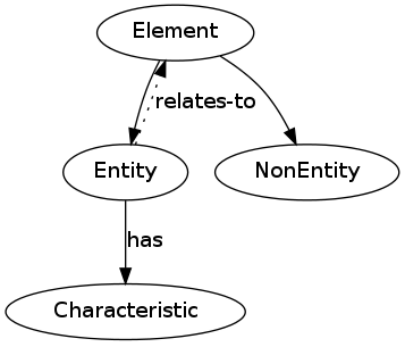

Fig. 5 eProfile Profile Ontology

trying to perform). This class is the most comprehensive of the ontology, and that will possibly be the most specialized by the trail managers.

The profile ontology (eOntoProfile), shown in Figure 5, is the ontology used to store the entity models, as well as the profile information that is inferred from the trail. For the ontology to be feasible within the proposed model, it must be a generic ontology that allows its classes to be extended by applications. In the figure, a relationship between classes is shown by a dotted line, and a specialization of a class is shown by a continuous line. The eOntoProfile is a basic ontology, extended by the applications so that they can assemble their own entity models.

The basic class of the ontology is the *Element* class. The class *Element* represents anything in the real world. It has the following children classes: (i) *Entity*, which represents an entity, that is, an element that has a profile associated with it, and (ii) *NonEntity*, which represents an element than doesn't have a profile associated with it. For example, a trail registers a event where a person enters a store, but the store does not have a profile in eProfile. In this case, the store is a *NonEntity*. This distinction is important because in

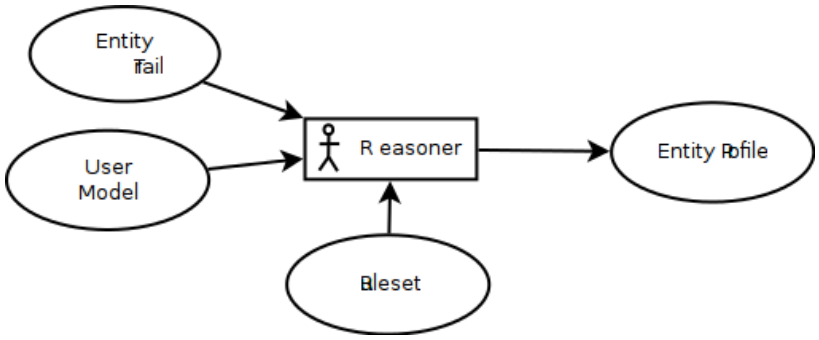

Fig. 6 Data flow for the entity profile inference

the first case, the eProfile has access to a wide range of information about the entity, the entities related to that entity, and so on.

The class *Characteristic* represent a aspect of the entity, and is the class that will be extended by applications to refer to any information related to that entity. These characteristics can be simple labels (such as name in the case of a person, or mileage in the case of a car) or more complex information (such as academic knowledge or latest routes traveled).

2.3 Rule Execution

The execution of rules is key to the functioning of eProfile. A inference rule is a instruction issued by an application, that runs on a entity trailset, resulting in a entity profilem according to a entity model previously registered by the application.

For a rule to be executed, the application must have previously registered with eProfile: (i) the rule itself, (ii) in an entity model, which serves as the basis for the entity profile format, and (iii) the trail of the entity on which the inference is performed. From these three pieces of information, the reasoner is able to generate a profile entity, according to Figure 6.

The rules are always defined by the application, and a rule is composed of the following set of information: (i) name, (ii) profile field that will be updated, (iii) the rule, written in OWL language (W3C, 2011), (iv) update type (the value can be overwritten or updated according to a formula), (v) update frequency, and (vi) expiration of the rule.

3 Implementation Aspects

The eProfile prototype was developed in Python (Rossum, 1995). Python is a generic, open source, object-oriented, high-level language, that supports multiple architectures. The databases are implemented using the SQLite3 database

(Hwaci, 2012). SQLite3 is an SQL transactional self-contained database, available in the public domain. The SQLite3 was chosen due to its good performance and the fact that it does not require a separate server for deployment.

Figure 6 shows the data flow for the entity profile inference. At the heart of this operation is the Reasoner agent, which uses an external inference engine to implement the rules. The reasoner used is FACT++, a propositional logic reasoner created as a platform for use of algorithms of semantic *tableaux* and optimization techniques (Tsarkov & Horrocks, 2006). This reasoner was chosen due to its good performance and support all functions used by the eProfile.

The knowledge representation language used for the storage and processing of ontologies is OWLW3C (2011), chosen because it is the most expressive ontology language defined or in definition for the Semantic Web (Daconta et al., 2003).

4 Evaluation Aspects

5 Related Works

6 Conclusion

References

Abowd, Gregory D., Anind K. Dey, Peter J. Brown, Nigel Davies, Mark Smith, & Pete Steggles 1999. Towards a Better Understanding of Context and Context-Awareness. In Proceedings of the 1st international symposium on Handheld and Ubiquitous Computing, HUC '99, pages 304–307, London, UK. Springer-Verlag.

Barbosa, Jorge Luis Victoria, Rodrigo Machado Hahn, Debora Nice Ferrari Barbosa, & Amarolinda Iara da Costa Zanel Saccol 2011. A ubiquitous learning model focused on learner interaction. International Journal of Learning Technology 2011, 6(1):62 – 83.

Carmagnola, Francesca, & Federica Cena 2009. User identification for cross-system personalisation. Inf. Sci., 179:16–32.

Daconta, Michael C., Leo J. Obrst, & Kevin T. Smith 2003. The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management. Wiley, Indianapolis.

Dey, A. 2001. Understanding and Using Context. Personal and Ubiquitous Computing, 6(1):4–7.

Franco, Laerte K., Joao H. Rosa, Jorge L.V. Barbosa, Cristiano A. Costa, & Adenauer C. Yamin 2011. MUCS: A model for ubiquitous commerce support. Electronic Commerce Research and Applications, 10(2):237 – 246. ¡ce:title¿Special Issue on Electronic Auctions: Strategies and Methods¡/ce:title¿.

Heckmann, Dominik 2005. Ubiquitous User Modeling. PhD thesis, Department of Computer Science, Saarland University, Germany.

Hwaci 2012. SQLite Home Page. `http://www.sqlite.org/`. Acessado em 05/05/2012.

IEEE 1991. IEEE Standard Computer Dictionary. A Compilation of IEEE Standard Computer Glossaries. IEEE Std 610, page 1.

Levis, Darci, Jorge L. V. Barbosa, Sérgio Crespo C. S. Pinto, & Debora Nice Ferrari Barbosa 2008. Aperfeiçoamento Automático do Perfil do Aprendiz em Ambientes de Educação Ubíqua. Revista Brasileira de Informática na Educação, 16(1).

Mulvenna, Maurice D., Sarabjot S. Anand, & Alex G. Büchner 2000. Personalization on the Net using Web mining: introduction. Commun. ACM, 43:122–125.

Rossum, Guido 1995. Python reference manual. Technical report, Amsterdam, The Netherlands, The Netherlands.

Satyanarayanan, M. 2001. Pervasive Computing: Vision and Challenges. IEEE Personal Communications, 8:10–17.

Silva, Jader, João Rosa, Jorge Barbosa, Débora Barbosa, & Luiz Palazzo 2010. Content distribution in trail-aware environments. Journal of the Brazilian Computer Society, 16:163–176. 10.1007/s13173-010-0015-1.

Silva, Jader, João Rosa, Jorge Barbosa, Débora N. F. Barbosa, & Luiz Antônio Moro Palazzo 2009. Content distribution in trail-aware environments. In Proceedings of the XV Brazilian Symposium on Multimedia and the Web, WebMedia '09, pages 15:1–15:8, New York, NY, USA. ACM.

Tsarkov, D., & I. Horrocks 2006. FaCT++ Description Logic Reasoner: System Description. In Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006), volume 4130 of *Lecture Notes in Artificial Intelligence*, pages 292–297. Springer.

Viviani, Marco, Nadia Bennani, & Elod Egyed-Zsigmond 2010. A Survey on User Modeling in Multi-Application Environments. In The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10, pages 111–116. IEEE.

W3C 2011. OWL Web Ontology Language Guide. `http://www.w3.org/TR/owl-guide/`. Acessado em 05/05/2012.

Weiser, Mark 1991. The Computer for the 21st Century. Scientific America, pages 94–104.

Weiser, Mark, & John Seely Brown 1996. Designing Calm Technology. PowerGrid Journal.